

Gray-box Wilson-Cowan: del plan del 10-09 a las primeras correcciones que funcionan

Informe de trabajo

11 de septiembre de 2026

El plan del 10 de septiembre afirmaba que la corrección neuronal fracasaba por falta de información y no de capacidad, y que darle la historia del comando la haría funcionar. Se implementó y se midió. El NRMSE sobre la planta con actuador bajó de 15,23 % a 9,12 % y el R^2 de la corrección rompió el techo teórico que ninguna función del estado observado podía superar. El barrido de capacidad dio un resultado más fino que el previsto: agrandar el filtro ayuda, agrandar la red o la ventana de historia perjudica. Quedan ocho experimentos corriendo. Este documento recorre lo hecho desde la escritura del plan, explica cada término y el motivo de cada decisión.

1 De qué trata el proyecto

El objetivo es identificar la dinámica de una población neuronal a partir de datos, combinando una ecuación conocida con una parte aprendida. La ecuación conocida es el **modelo de Wilson-Cowan**, un modelo de masa neuronal que describe dos poblaciones acopladas, una excitatoria y una inhibitoria, mediante dos ecuaciones diferenciales ordinarias. Sus variables de estado son E , la actividad de la población excitatoria, e I , la de la inhibitoria. El estado del sistema es el vector $x = (I, E)^\top$. Las entradas externas son P y Q , los estímulos que recibe cada población.

El modelo tiene diez parámetros que se agrupan en el vector $\beta \in \mathbb{R}^{10}$: cuatro pesos de acoplamiento (w_{EE} , w_{EI} , w_{IE} , w_{II}), dos constantes de tiempo (τ_e , τ_i), dos pendientes de la función de activación (a_e , a_i) y dos umbrales (θ_e , θ_i). El nombre β se eligió porque la letra θ , la habitual para un vector de parámetros, colisiona con los umbrales θ_e y θ_i del propio modelo.

1.1 Qué es un gray-box y por qué no un black-box

Un **black-box** aprende la dinámica entera con una red neuronal, sin suponer nada. Reproduce bien y no explica nada: los pesos aprendidos no corresponden a ninguna cantidad física. Un **white-box** escribe la ecuación completa a mano y ajusta solo sus parámetros. Explica todo lo que acierta, pero si la planta real tiene algo que la ecuación no contempla, ese algo no se puede representar y el error queda.

El **gray-box** es la combinación:

$$\dot{x} = \underbrace{f_{WC}(x, P, Q; \beta)}_{\text{física escrita a mano}} + \underbrace{g_\varphi(\cdot)}_{\text{red neuronal}} \quad (1)$$

El primer término aporta la estructura conocida y sus parámetros conservan significado físico. El segundo, con parámetros φ , absorbe lo que la estructura no cubre. La promesa es tener las dos cosas: reproducción de un black-box e interpretabilidad de un white-box.

El problema central del proyecto es que esa promesa venía sin cumplirse. En las dos únicas configuraciones que reproducían bien la dinámica, la red estaba literalmente apagada (`use_correction=False`) y todo el trabajo lo hacía física escrita a mano. Eso es lo que disparó el plan del 10 de septiembre.

1.2 La planta simulada

Los datos son de simulador, no experimentales. Se genera una trayectoria con Wilson-Cowan más una **perturbación** que el modelo reducido no contempla, y después se pide identificar

el sistema usando solo el modelo reducido más la corrección. La diferencia entre la dinámica verdadera y la que el modelo reducido puede representar se llama Δf , y es exactamente lo que g tendría que aprender.

Se usan dos plantas, cada una con una sola perturbación activa, para atacar la dificultad de a una por vez:

- **refrac1: refractariedad.** Después de disparar, una neurona no puede volver a hacerlo de inmediato. Se modela con un factor $(1 - r x)$ que reduce la respuesta cuando la actividad es alta. Es una función del estado actual y de nada más.
- **act1: actuador.** El estímulo que llega a la población no es el comando u que se ordena, sino una versión filtrada y saturada de él. En concreto un **sistema de Wiener**, que es un filtro lineal seguido de una no linealidad sin memoria:

$$\frac{dP_{\text{lag}}}{dt} = \frac{P - P_{\text{lag}}}{\tau}, \quad \tau = 1 \text{ ms}, \quad P_{\text{efectivo}} = A \tanh\left(\frac{P_{\text{lag}}}{A}\right), \quad A = 3. \quad (2)$$

Acá está la dificultad: el estímulo efectivo depende del *pasado* del comando, no solo de su valor actual.

1.3 Las métricas y qué mide cada una

- **NRMSE de corrida libre** (*free-run rollout*). Se le da al modelo identificado el estado inicial y la secuencia de estímulos, y se lo deja integrar la trayectoria entera sin volver a mirar los datos. Se compara contra la trayectoria verdadera y se normaliza. Mide reproducción de la dinámica. Es la métrica principal.
- **R^2 de la corrección.** Compara, muestra a muestra, lo que la corrección aprendida $\hat{g}$ produce contra el término omitido verdadero Δf :

$$R^2 = 1 - \frac{\sum \|\hat{g} - \Delta f\|^2}{\sum \|\Delta f - \bar{\Delta f}\|^2} \quad (3)$$

Mide recuperación de física, no reproducción. Un modelo puede copiar la trayectoria con R^2 pésimo, si la corrección compensa por un camino distinto del verdadero.

- **Error de parámetros.** Distancia relativa entre el β identificado y el β verdadero del simulador. Es el control que dice si la corrección está deformando la física en vez de complementarla.
- **Fracción de redundancia.** Cuánto de lo que hace la corrección lo podría haber hecho un cambio de parámetros. El punto de comparación no es cero: el Δf verdadero ya puntúa 0,67, y una red con pesos aleatorios puntúa 0,47 solo por ser una función suave del estado.

1.4 Cómo se entrena

Se usa **multiple shooting**: la trayectoria se parte en ventanas cortas y cada ventana se integra desde el estado observado en su comienzo, en vez de integrar los 4000 pasos de un tirón. El motivo es de condicionamiento numérico. Integrar una trayectoria larga y derivar respecto de los parámetros hace que el gradiente explote o se desvanezca, porque el error se propaga multiplicativamente a lo largo de la integración. Reiniciando cada 100 pasos el problema queda mucho mejor planteado.

Después de la fase de descenso por gradiente viene una fase de **L-BFGS**, un método de segundo orden que converge mucho más rápido cerca del mínimo pero que necesita arrancar razonablemente cerca.

2 El plan del 10-09 y su razonamiento

2.1 El hallazgo que lo disparó

Preparando el póster de la Jornada se verificó en el código que en las dos correcciones que funcionaban, la red estaba apagada. La primera, *forma exacta*, agrega la refractariedad al backbone como tres parámetros físicos más. La segunda, *estado de filtro*, aumenta el estado con dos variables que siguen la dinámica del actuador, y son doce parámetros en total, sin ninguna red. Las dos reproducen bien. Ninguna de las dos usa g .

Eso vacía el argumento del gray-box. Si lo único que funciona es escribir la ecuación a mano, el proyecto es un white-box con pasos extra.

2.2 El teorema y qué dice exactamente

El **Teorema de Aproximación Universal** dice que una red neuronal suficientemente ancha puede aproximar cualquier función continua *de sus entradas*, con la precisión que se quiera. La palabra que hace todo el trabajo es *de sus entradas*.

Sobre act1 , Δf depende del pasado del comando. Una red que solo ve (I, E) no puede representarla, porque Δf *no es una función* de (I, E) : para un mismo estado observado, Δf toma valores distintos según lo que haya pasado antes. Esto no es una limitación de capacidad. Es una limitación de información, y ninguna cantidad de capas la arregla.

Está medido. El **oráculo**, que es la mejor función posible de (I, E) ajustada directamente contra Δf sin ninguna restricción de optimización, llega a $R^2 = -0,11$ sobre la planta nominal. Ese es un techo duro para toda variante que le dé a la red solo el estado.

2.3 La decisión: memoria en la entrada, no en el estado

Hay dos formas de darle memoria a un modelo. La primera es un estado interno propio que el modelo infiere y propaga, que es la **latent ODE**. Ya estaba medida y fracasó: $R^2 = -0,51$, la peor de todas y la más cara. El motivo es que ese estado latente hay que inferirlo de los datos, que es un problema mucho más difícil que el original.

La segunda es **memoria en la entrada**: se le pasan a la red las últimas K muestras del comando, que son conocidas y no hay que inferir nada. En identificación de sistemas esto es un modelo **NARX** o un **FIR** sobre la entrada. Con la historia adentro, Δf *sí* es una función continua de la entrada de la red, y el teorema aplica sin asteriscos.

2.4 Una corrección al propio plan

La primera versión fijaba $K = 100$ muestras, justificado como 5τ . Eso es darle física a la red, aunque sea poca: usa el valor de τ , que es justo lo que tendría que descubrir. Se reemplazó por $K = 400$ muestras (20 ms), muy por encima de cualquier constante de tiempo plausible, dejando que el núcleo aprendido decaiga solo. El largo efectivo de la memoria pasa de ser un supuesto a ser una medición.

3 Lo que se midió

Todas las corridas usan ventana de 100 pasos, 1500 épocas, semilla 0, y el mismo dataset, para que sean comparables entre sí.

3.1 Eje 1: la historia del comando

corrida	qué ve g	pesos	NRMSE	R^2	err. param
e2_wb	nada, sin corrección	0	15,23	-0,006	30,7
e2_B	(I, E)	1 218	15,40	-1,875	34,4
e2_A	(I, E, P, Q) instantáneo	1 346	15,18	-0,714	32,9
e2_H100	$(I, E) + 100$ retardos	7 618	12,92	+0,250	26,3
e2_H400	$(I, E) + 400$ retardos	26 818	11,48	+0,146	22,7
e3_K400	ídem, filtrado por un FIR	4 546	10,16	-0,640	16,7
e3_K400_f8	ídem, filtro de 8 canales	7 874	9,12	+0,127	21,4
e2_lag2	forma estructural, sin red	12	2,96	+0,942	5,1

Lo que confirma la predicción central. Las tres variantes sin memoria empatan o empeoran respecto de no corregir nada. Las variantes con memoria le ganan al white-box por primera vez, y el R^2 se vuelve positivo, rompiendo el techo de -0,11 que ninguna función del estado observado podía superar. La información faltante era la que se predijo.

Lo que no se predijo. Las tres métricas se mueven en direcciones distintas al aumentar el campo receptivo. El NRMSE mejora de forma monótona ($12,92 \rightarrow 11,48 \rightarrow 10,16$) y el error de parámetros también ($26,3 \rightarrow 22,7 \rightarrow 16,7$), pero el R^2 se degrada y termina negativo ($+0,250 \rightarrow +0,146 \rightarrow -0,640$).

La lectura habitual de un R^2 negativo es que la corrección absorbe física en el lugar equivocado. Bajo esa lectura el β identificado se tendría que degradar, y está mejorando casi a la mitad del error. Las dos cosas no pueden ser ciertas a la vez.

La explicación más plausible es que el R^2 se calcula punto a punto contra Δf , mientras que el NRMSE mide el efecto acumulado sobre la trayectoria. Un término que difiere punto a punto pero produce el mismo efecto dinámico puntúa mal en el primero y bien en el segundo. La sección siguiente da evidencia directa de por dónde va esa diferencia.

3.2 Eje 1.3: por qué un filtro y no más entradas crudas

La variante H le da a la red los $2K$ retardos crudos. Con $K = 400$ son 802 entradas, y solo la primera capa densa pesa 26 mil parámetros. Esa capa aprende un peso independiente por cada retardo, lo cual no supone nada pero cuesta mucho y deja mucho margen para ajustar cualquier cosa.

La variante K interpone un **FIR** (*finite impulse response*, un filtro de respuesta finita al impulso) de cuatro canales antes de la red. Un FIR calcula $\sum_k w_k u(t-k)$, que es una convolución con un núcleo libre. No se le dice qué forma tiene el núcleo. Lo único que se supone es **invarianza temporal**, o sea que el efecto de un retardo dado no cambia según el instante absoluto. Eso es un supuesto de estructura, no de física.

El resultado es 4546 parámetros contra 26818, casi seis veces menos, y mejor NRMSE. Y hay un beneficio adicional que ninguna otra variante ofrece: el núcleo aprendido se puede graficar y comparar con la física verdadera.

3.3 Eje 1.4: dónde agregar capacidad y dónde no

Con la variante K funcionando queda una pregunta que el plan dejaba condicionada: si lo que falta es capacidad. Hay tres perillas independientes y se movió una por vez desde e3_K400.

corrida	qué cambió	pesos	NRMSE	R^2	err. param
e3_K400	referencia	4 546	10,16	-0,640	16,7
e3_K800	doble ventana de historia	7 746	12,48	-0,397	16,4
e3_K400_h64	doble ancho de la red	7 938	10,98	-0,861	25,7
e3_K400_f8	doble rango del filtro	7 874	9,12	+0,127	21,4

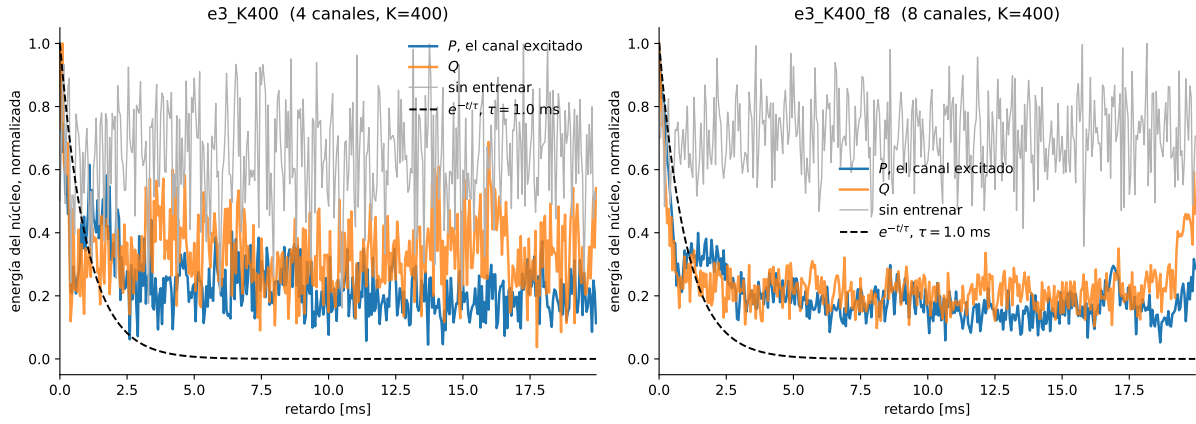


Figura 1: Energía del núcleo FIR aprendido en función del retardo, sobre `act1`. Izquierda, `e3_K400`, cuatro canales. Derecha, `e3_K400_f8`, ocho canales. En los dos casos $K = 400$ muestras, o sea 20 ms de campo receptivo. La energía es la norma sobre los canales para cada retardo, normalizada a su máximo; los canales por separado no son comparables porque la red que sigue puede reescalar cualquiera de ellos sin cambiar la función. La curva gris es el mismo filtro sin entrenar, y sirve de control para distinguir estructura de ruido. La curva negra punteada es la respuesta al impulso del actuador verdadero, $e^{-t/\tau}$ con $\tau = 1,0$ ms.

Las tres tienen un número de parámetros casi idéntico, entre 7,7 y 8 mil, así que la comparación aísla *dónde* se agrega la capacidad y no *cuánta*.

Más ventana de historia perjudica. Con $K = 800$ el campo receptivo llega a 40 ms y la física ocupa los primeros 5. Los 795 retardos restantes son capacidad libre para ajustar lo que no hay que ajustar, y el NRMSE sube de 10,16 a 12,48.

Más ancho de red perjudica más. El NRMSE empeora poco pero el error de parámetros salta de 16,7 a 25,7. Es la ambigüedad entre β y g apareciendo limpia: con más capacidad la red absorbe lo que los parámetros deberían explicar. Y no es un canje, porque las dos métricas empeoran a la vez.

Más canales de filtro es lo único que ayuda, y confirma una predicción. Al ver el R^2 negativo de `e3_K400` se había planteado que con cuatro canales el filtro solo puede formar un subespacio de rango 4 de la historia, y que si esa era la restricción, ampliarlo tenía que recuperar el R^2 . Pasó de $-0,640$ a $+0,127$, y el NRMSE bajó al mejor valor de toda la serie.

Queda una tensión sin explicar. La corrida de ocho canales reproduce mejor y recupera más física, pero identifica peor los parámetros (21,4 contra 16,7). Las tres métricas no se ordenan de la misma manera en ninguna de las cuatro corridas, y eso todavía no tiene una lectura única.

Una advertencia sobre los tiempos. El campo de duración es reloj de pared, no tiempo de cómputo. La corrida de ocho canales figura con 938 minutos porque arrancó a las 23:55 y terminó a las 15:34 del día siguiente, con la máquina probablemente dormida en el medio. No se puede verificar después del hecho, así que esa cifra no se debe comparar con las otras.

3.4 El núcleo aprendido

El FIR es la única variante que permite mirar lo que la corrección aprendió sobre el tiempo, porque sus pesos son directamente el peso que se le da a cada retardo. Se comparan contra la respuesta al impulso del actuador verdadero, que el entrenamiento nunca vio.

Cuatro lecturas, en orden de importancia:

1. **El filtro encontró el pico en retardo cero.** La relación entre la cola (más allá de 10 ms) y el pico pasó de 1,27 sin entrenar a 0,20 con cuatro canales y 0,16 con ocho. El

entrenamiento concentró energía en los retardos cortos, que es donde está la física. Eso es aprendizaje real, no ruido.

2. **Con ocho canales la caída inicial sigue a la exponencial.** Es visible en el panel derecho hasta unos 2 ms, y no lo es en el izquierdo. La fracción de energía en los primeros 5 ms sube de 34,0 % a 37,9 %, contra 25 % de un filtro sin entrenar y 99,3 % de la exponencial.
3. **Pero ninguno suprime la cola, y el núcleo no es una exponencial sola.** Dos descriptores del mismo núcleo dan respuestas distintas, y esa discrepancia es el resultado. El tiempo hasta caer a $1/e$ del pico, que no supone ninguna forma, da 0,50 ms con ocho canales (0,30 con cuatro) contra 1,0 verdadero. El ajuste de una exponencial sobre el tramo previo al piso da 4,67 ms (10,88 con cuatro). Uno queda por debajo del valor verdadero y el otro muy por encima, porque la curva tiene una caída rápida seguida de un piso plano y ninguna exponencial describe las dos cosas. Lo defendible es que la componente rápida está en el orden de magnitud correcto, dentro de un factor dos, y que ampliar el filtro la acercó. No se puede afirmar que se recuperó τ .
4. **El piso da una explicación concreta del R^2 negativo con cuatro canales.** Los pesos de retardo largo quedan casi libres, porque la función de costo depende muy poco de ellos. Su contribución a $\hat{g}$ es una componente esencialmente arbitraria que no se parece a Δf y que el R^2 castiga punto a punto, mientras que el optimizador ya la ajustó para que no dañe la trayectoria. Es consistente con que $K = 800$, que duplica esos pesos libres, empeore el NRMSE.

Esto convierte una anomalía en un experimento. Si el diagnóstico es correcto, penalizar los pesos del filtro tiene que levantar el R^2 y bajar el piso sin arruinar el NRMSE. Quedó implementado (`--wd-fir`) y encolado con dos intensidades.

4 Los errores encontrados

Cuatro fallas de cableado aparecieron en esta tanda. Todas comparten una forma: una lista de variantes escrita en más de un lugar del código, que se actualizó en uno y no en los otros. Vale registrarlas porque tres de ellas no daban síntoma.

1. **La variante H no estaba en la lista de argumentos válidos.** Las dos corridas murieron al instante. Es la falla barata: se ve enseguida.
2. **El cálculo del error open-loop no recibía el largo de la historia.** La red terminaba con cuatro entradas en vez de $2 + 2K$.
3. **El diagnóstico final evaluaba g sobre 3000 puntos sueltos**, que por definición no tienen historia asociada. Se cambió para que use las propias ventanas de entrenamiento, que sí la tienen.
4. **El filtro FIR nunca se entrenó.** Esta es la cara. La corrida `e3_K400` reportó NRMSE 20,90 %, peor que no corregir nada, porque el filtro era módulo hermano de `self.g` y el optimizador arma su grupo con `model.g.parameters()`. Se quedó 1500 épocas en su inicialización aleatoria, y la red trabajó sobre cuatro proyecciones fijas y arbitrarias de la historia. Cuarenta y tres minutos de cómputo para un resultado que medía otra cosa.

4.1 Qué se cambió para que no se repita

- **El filtro pasó a vivir dentro de g.** Seis lugares del repositorio toman `model.g.parameters()` como “toda la corrección”. Hacer al filtro hijo de `g` arregla los seis a la vez, en lugar de agregarlos a mano en uno solo y dejar cinco mal.
- **Un guarda en la función de entrenamiento.** Antes de construir el optimizador compara los parámetros entrenables del modelo contra los que están en sus grupos, y aborta si sobra alguno. Un parámetro fuera del optimizador no da síntoma: la corrida termina, reporta y guarda un número plausible que mide otra cosa.
- **El checkpoint se guarda antes de evaluar.** Con el orden anterior, cualquier error en las últimas líneas del script costaba la corrida entera.
- **El test de variantes chequea el gradiente por parámetro.** Antes sumaba sobre toda la corrección, y esa suma daba positiva igual porque la red sí recibía gradiente. Eso obligó a un detalle: la última capa arranca en cero a propósito, así que en el primer paso ninguna capa anterior recibe gradiente y el chequeo no distinguiría un huérfano de uno sano. Se la despierta antes de medir.
- **Las decisiones de “necesita historia” salen de una sola función.** Es la causa raíz común de tres de los cuatro errores.

5 El ruido, que hasta acá está apagado

5.1 Por qué no estaba

Todo el escalado corre sobre datos limpios. Los dos datasets se generan con `noise_std = 0.0`, a propósito. Con ruido encima no se puede distinguir si el residuo que la corrección tiene que aprender es el término omitido o la medición, y el R^2 de la corrección dejaría de significar algo. Los 10,16 % de NRMSE son sobre trayectorias exactas.

5.2 Lo que sí está medido

En una línea de trabajo anterior, sobre el white-box de diez parámetros. Error máximo de parámetro, con media móvil aplicada en los cuatro niveles:

σ	10 parámetros	subconjunto de 4
0	0,8 %	0,1 %
0,01	4,5 %	0,6 %
0,05	19,3 %	1,2 %
0,10	41,3 %	8,9 %

La mitigación fue promediar antes de ajustar, con una media móvil de siete muestras hasta $\sigma = 0,05$ y de once por encima. Lo que separa las dos columnas es fijar los parámetros no identificables en lugar de pelearlos, y el efecto es grande. El problema nunca fue el ruido solo, sino el ruido sobre parámetros mal condicionados.

Hay un caso que no se mitiga y sirve de control negativo. El **ruido de proceso**, el que entra dentro de la ecuación diferencial en vez de sumarse a la medición, da R^2 del oráculo de 0,12 con el estado y 0,11 agregando el comando. No es función de nada, así que ninguna corrección lo puede aprender, ni con historia ni con capacidad.

5.3 La hipótesis que se va a probar

La variante H le da a la red 802 entradas y 26 mil pesos en la primera capa, que es exactamente la configuración que ajusta ruido. El FIR promedia K muestras, y eso es un filtro pasabajos por construcción. Si el FIR se degrada menos al subir el ruido, el argumento a favor de la convolución deja de ser solo el costo.

Se prepararon `act1_n01` ($\sigma = 0,01$, el 5,7 % del desvío de E) y `act1_n05` ($\sigma = 0,05$, el 27,5 %). El ruido se suma al dataset ya generado en vez de regenerar las trayectorias, por dos motivos: la trayectoria queda idéntica a la limpia, así que las corridas son comparables muestra a muestra, y el Δf de referencia sigue calculado sobre la trayectoria exacta, que es lo único que hace válido el R^2 .

6 Qué quedó corriendo

Las cuatro del eje 1.4 ya terminaron y están arriba. Quedan ocho, en secuencia y nunca en paralelo. La bitácora ya registra que cuatro carriles sobre ocho núcleos convirtieron corridas de 25 minutos en corridas de 25 horas.

corrida	planta	qué contesta
e4_Sg	refrac1	Cuánto se tapan entre sí los parámetros y la red, sobre la planta donde la forma exacta está completa. Si la red no aprende nada, su magnitud queda cerca de cero.
e4_S2ver	refrac1	Verificación del dataset. Tiene que reproducir NRMSE 1,70 y R^2 0,964.
e4_wb_n05, e4_K400_n05, e4_H400_n05	act1_n05	El eje 6 con $\sigma = 0,05$, con el white-box como referencia.
e4_K400_n01, e4_H400_n01	act1_n01	Ídem con $\sigma = 0,01$.
e5_K400_wd1e-4, e5_K400_wd1e-3	act1	Si penalizar los pesos del filtro levanta el R^2 sin arruinar el NRMSE.

7 Dónde está el proyecto

La escala honesta: la forma estructural sigue ganando por lejos, 2,96 contra 9,12 en NRMSE y 0,942 contra 0,127 en R^2 . El camino generalizable funciona y rompió el techo teórico, pero todavía está a un tercio del camino en reproducción. Escribir la ecuación a mano sigue siendo mucho mejor que aprenderla.

Lo que cambió respecto del 10 de septiembre son tres cosas que antes no existían. Hay un camino que mejora al agrandarlo por una dirección concreta, el rango del filtro. Hay una explicación medible de por qué todavía no llega, que es el piso de pesos libres en los retardos largos. Y hay una predicción que se hizo antes de medirla y se cumplió, la del rango del filtro sobre el R^2 .

7.1 Lo que queda decidido

La prioridad pasó a ser el NRMSE. El R^2 de la corrección se reporta siempre y no bloquea nada. El criterio de aceptación es NRMSE por debajo de 9,12 %, con el white-box (15,23) como piso y la forma estructural (2,96) como referencia de cuánto falta.

El control que mantiene honesta esa prioridad es el error de parámetros. Si la corrección mejorara el NRMSE absorbiendo física en el lugar equivocado, el β identificado se degradaría. En la serie con memoria bajó de 30,7 a 16,7 junto con el NRMSE, así que la mejora es real y no un canje. La corrida de ocho canales rompe esa correlación: mejora el NRMSE y el R^2 pero

empeora el error de parámetros a 21,4. Vale la pena vigilarlo, porque una variante que gane NRMSE devolviendo el error de parámetros al nivel del white-box no sirve para el propósito del gray-box.

7.2 Lo que queda pendiente de una decisión

- **Los datos reales.** Para probar cualquier variante sobre una grabación hay que estimar el estado inicial de cada ventana, porque lo que se observa no es (I, E) sino una proyección escalar $s = c(E - I)$. La recomendación es optimizar solo el estado inicial con los parámetros congelados, pero es una decisión de método y no se tomó.
- **La revisión del repositorio limpio** antes de hacerlo público. Ya aparecieron tres afirmaciones falsas en su README, así que conviene asumir que hay más.